

Back-End, Databases & Final Project

Days 12–20 | Full-Stack Integration, PHP 8, MySQL PDO, Dynamic Fetch API & Plant Care Guide Capstone

1. Dynamic Frontend

Client validation, JSON object handling, debounced live search via JS Fetch API.

2. PHP & Database

PHP control flows, POST/GET handling, MySQL relational schema, and secure PDO CRUD.

3. Capstone Guide

Complete full-stack architecture, API endpoints, debugging, and project demonstration.

Day 12: JavaScript Validation, JSON & Fetch/API Basics

Day 12

Client-side validation improves user experience by intercepting errors prior to HTTP transmission, while **Fetch API** enables asynchronous background server queries.

Validation & JSON Handling:

```
function validatePlant(data) {
  const errors = [];
  if (!data.name || data.name.trim().length < 2) {
    errors.push("Name must be 2+ chars.");
  }
  if (!data.waterDays || data.waterDays < 1) {
    errors.push("Watering interval must be > 0.");
  }
  return { isValid: errors.length === 0, errors };
};

// JSON Parsing & Stringification
const obj = { name: "Monstera", waterDays: 7 };
const jsonStr = JSON.stringify(obj);
const parsed = JSON.parse(jsonStr);
```

Async Fetch Endpoint Query:

```
async function fetchPlants(query = "") {
  try {
    const res = await fetch(
      `/api/plants.php?search=${encodeURIComponent(query)}`
    );
    if (!res.ok) throw new Error("Network error");
    return await res.json();
  } catch (err) {
    console.error("Fetch failure:", err);
    return [];
  }
}
```

Assignment 8: Add Plant Search Functionality (Debounced Live Search)

```
const searchBar = document.getElementById("searchBar");
let debounceTimer;

searchBar.addEventListener("input", (e) => {
  clearTimeout(debounceTimer);
  const q = e.target.value.trim();

  // Debounce API calls by 300ms to reduce database load
  debounceTimer = setTimeout(async () => {
    const plants = await fetchPlants(q);
    renderSearchResults(plants);
  }, 300);
});
```

Day 13: PHP Introduction, Variables, Conditions, Loops & Functions

Day 13

PHP is a server-side scripting language embedded within HTML to render dynamic content before returning responses to the browser.

```
<?php
// Basic Variables and Associative Arrays
$siteTitle = "My Plant Care Guide";
$plants = [
    ["name" => "Monstera", "category" => "Indoor", "water_days" => 7],
    ["name" => "Snake Plant", "category" => "Low Light", "water_days" => 14]
];

// Helper Function for Date Calculations
function calculateWateringDate(int $days): string {
    $dt = new DateTime();
    $dt->modify("+{$days} days");
    return $dt->format('Y-m-d');
}
?>
```

Day 14: PHP Forms, GET/POST & Form Validation

Day 14

Method	Data Transmission	Primary Use Cases & Security Profile
\$_GET	URL query string parameters (?q=plant)	Non-sensitive data, search filtering, bookmarkable dynamic links. Limit ~2KB.
\$_POST	HTTP Request Body (Hidden from URL)	Form submissions, database inserts, passwords, sensitive payloads. High limit.

Plant-Care Request Processing (care_request.php)

<?php
\$errors = [];
\$success = "";

if (\$_SERVER["REQUEST_METHOD"] === "POST") {
 // 1. Input Sanitization
 \$email = filter_var(trim(\$_POST["user_email"]) ?? "", FILTER_SANITIZE_EMAIL);
 \$issue = htmlspecialchars(trim(\$_POST["issue"]) ?? "");

 // 2. Validation Checks
 if (empty(\$email) || !filter_var(\$email, FILTER_VALIDATE_EMAIL)) {
 \$errors[] = "Please enter a valid email address.";
 }
 if (strlen(\$issue) < 10) {
 \$errors[] = "Issue description must be at least 10 characters.";
 }

 // 3. Process Request
 if (empty(\$errors)) {
 \$success = "Care request received successfully!";
 }
}
?>

Day 15: Server Stack & MySQL Database Setup

Day 15

XAMPP packages Apache (Web Server), MySQL/MariaDB (Database Engine), and PHP. **phpMyAdmin** serves as the web management console for visual database configuration.

Assignment 9: Plant Care Database & Table Schema Creation Script

```
CREATE DATABASE IF NOT EXISTS `plant_care_db`  
DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
  
USE `plant_care_db`;  
  
CREATE TABLE IF NOT EXISTS `plants` (  
  `id` INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  `name` VARCHAR(100) NOT NULL,  
  `scientific_name` VARCHAR(150) DEFAULT NULL,  
  `category` VARCHAR(50) NOT NULL,  
  `water_frequency_days` INT NOT NULL,  
  `light_requirement` VARCHAR(50) NOT NULL,  
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

Day 16: PHP + MySQL Connectivity (PDO), INSERT & SELECT

Day 16

PHP Data Objects (PDO) provides a secure, object-oriented layer for database queries, preventing SQL Injection using prepared statements.

1. Database Connection (config/db.php):

```
<?php  
$host = "localhost";  
$dbname = "plant_care_db";  
$user = "root";  
$pass = "";  
  
try {  
    $pdo = new PDO(  
        "mysql:host={$host};dbname=  
{$dbname};charset=utf8mb4",  
        $user, $pass, [  
            PDO::ATTR_ERRMODE =>  
PDO::ERRMODE_EXCEPTION,  
            PDO::ATTR_DEFAULT_FETCH_MODE =>  
PDO::FETCH_ASSOC  
        ]  
    );  
} catch (PDOException $e) {  
    die("DB Error: " . $e->getMessage());  
}  
?>
```

2. Prepared INSERT & SELECT:

```
<?php  
require_once 'config/db.php';  
  
// Prepared INSERT  
$stmt = $pdo->prepare("  
    INSERT INTO plants (name, category,  
    water_frequency_days)  
    VALUES (:name, :cat, :water)  
");  
$stmt->execute([  
    ':name' => 'Monstera',  
    ':cat' => 'Indoor',  
    ':water' => 7  
]);  
  
// SELECT All Records  
$query = $pdo->query("SELECT * FROM plants");  
$plants = $query->fetchAll();  
?>
```

Operation	SQL Command	PDO Method & Logic
Create	INSERT INTO plants...	\$stmt->prepare() with named parameters.
Read	SELECT * FROM plants WHERE...	\$stmt->fetchAll() or fetch() for single item.
Update	UPDATE plants SET... WHERE id = :id	\$stmt->execute([':id' => \$id, ...])
Delete	DELETE FROM plants WHERE id = :id	\$stmt->execute([':id' => \$id])

Update Record (edit_plant.php):

```
<?php
$id = (int)$_GET['id'];
$stmt = $pdo->prepare("
    UPDATE plants
    SET name = :name, water_frequency_days =
:water
    WHERE id = :id
");
$stmt->execute([
    ':name' => $_POST['name'],
    ':water' => (int)$_POST['water_days'],
    ':id' => $id
]);
?>
```

Delete Record (delete_plant.php):

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $id = (int)$_POST['id'];
    $stmt = $pdo->prepare(
        "DELETE FROM plants WHERE id = :id"
    );
    $stmt->execute([':id' => $id]);
    header("Location: index.php?
status=deleted");
}
?>
```

Modular Directory Tree Structure

plant-care-guide/
├── config/
│ └── db.php # PDO connection
├── api/
│ └── get_plants.php # Dynamic JSON API
├── assets/
│ ├── css/style.css # Custom styles
│ └── js/app.js # Fetch API DOM script
├── index.php # Main dashboard UI
└── edit_plant.php # Form editing view

RESTful API Endpoint (api/get_plants.php):

```
<?php
header('Content-Type: application/json');
require_once '../config/db.php';

$q = trim($_GET['search'] ?? '');
if (!empty($q)) {
    $stmt = $pdo->prepare("SELECT * FROM plants
WHERE name LIKE :q");
    $stmt->execute([':q' => "%{$q}%"]);
} else {
    $stmt = $pdo->query("SELECT * FROM plants
ORDER BY id DESC");
}
echo json_encode($stmt->fetchAll());
?>
```

Day 19: Project Testing, Security & Responsive Polish

Day 19

SQL Injection Prevention

Always use PDO parameterized bindings. Never concatenate un-sanitized client input into SQL query strings.

Cross-Site Scripting (XSS) Guard

Wrap output rendered in HTML templates with `htmlspecialchars($str, ENT_QUOTES, 'UTF-8')`.

Assignment 10: Complete QA Test Suite Matrix

- ☐ DB Connection verified with error handling.
- ☐ Create: Form inserts new plant to database.
- ☐ Read: Dashboard populates from MySQL table.
- ☐ Update & Delete: Record modifications work cleanly.
- ☐ Live Search: Debounced Fetch calls update dynamic UI.
- ☐ Layout: Responsive on Mobile, Tablet & Desktop.

Day 20: Final Project Demo & Presentation Strategy

Day 20

Prepare a 10-minute capstone walkthrough structure for presentation day:

1. Overview (2 mins)

Define problem statement, target audience, and key features of the Plant Care Guide.

2. Architecture (3 mins)

Explain full-stack pipeline: JS Fetch API → PHP REST API → MySQL Database (PDO).

3. Live Demo (5 mins)

Demonstrate live search, dynamic form validation, record creation, editing, and deletion.

Congratulations on Completing Module 3!

You have built a full-stack, database-driven web application with security, API architecture, and dynamic JavaScript user interactions.